As fontes abordam a transformação da engenharia de software através da integração entre a **reimaginação das entrevistas técnicas**, os **princípios da Manufatura Enxuta (*Lean*)** e a **atuacao com Inteligência Artificial**:

1. A Reimaginação das Entrevistas e do Recrutamento

- **Obsolescência do LeetCode:** Testes algorítmicos tradicionais perderam o poder de diferenciar candidatos, pois modelos de linguagem modernos atingem taxas de sucesso expressivas (31,68% a 43,36% em desafios difíceis) e os candidatos contam com assistentes em tempo real 1, 2. Além disso, estudos empíricos mostram que horas acumuladas em treino estático de LeetCode possuem correlação nula (\\(r=0.0\\)) com a prontidão real para entrevistas 1.
- **Deslocamento do Foco:** O centro de gravidade da seleção mudou do *"como construir"* (sintaxe e digitação) para o *"o que"* e *"por que construir"* (formulação do problema, arquitetura e geração de valor) 3, 4.
- **O Modelo *Plan / Build / Review*:** A avaliação técnica moderna reordenou a distribuição de seus pesos: o **Planejamento inicial** responde por 40% da nota, a **Construção guiada por IA** por 20%, e a **Revisão/Auditoria crítica** (como testes e *code review* de *Pull Requests*) por 40% 5-9.

2. A Conexão com a Manufatura Enxuta (*Lean Software Engineering*)

- **Eliminação de Desperdício (*Waste*):** Avaliar engenheiros por decorar rotinas isoladas ou produzir volumes de código sem validação de mercado gera um desperdício massivo de tempo e talento humano 10, 11.
- **Aprendizado Validado:** Na filosofia *Lean Startup*, o progresso real não é medido pela quantidade de código escrito, mas pelo **aprendizado validado** — a capacidade de testar hipóteses e descobrir rapidamente o que o cliente realmente precisa 11-14.
- **Lotes Pequenos e Ciclos Rápidos:** A aplicação de pequenos lotes de alteração e o uso de implantação contínua (*Continuous Deployment*) reduzem os riscos de grandes falhas e permitem validar cada mudança no sistema de forma rápida e controlada 15-17.

3. Fluência em IA e Supervisão Humana (*Human-in-the-Loop*)

- **Regência e Auditoria:** A IA atua como um acelerador de execução, mas não toma decisões de arquitetura nem analisa *trade-offs* de negócios; cabe ao desenvolvedor atuar como um auditor rigoroso contra falhas de segurança e código alucinado 18-21.
- **Prompts Restritivos:** Para obter código confiável da IA, é essencial usar engenharia de prompt estruturada, definindo restrições negativas explícitas (ex.: *"não altere a assinatura da função"*), dividindo tarefas complexas em etapas e estabelecendo critérios claros de conclusão (*done criteria*) 22-26.

💡 **Gostaria de explorar como simular na prática uma sessão de *Code Review Gauntlet* ou prefere ver como estruturar um plano de estudos focado no modelo *Plan / Build / Review*?**